



ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA



Aplicaciones en Tecnologías Web

TEMA: Desarrollar una Aplicación Web en PHP con el modelo MVC para la gestión de Libros y Autores.

Integrantes:

Alexis Damián Morales Cuasquer
Jessica Estefania Sánchez Ugsiña
Melanie Abigail Talavera

NRC:1382

Contenido

1. Objetivos	3
1.1. Objetivo General.....	3
1.2. Objetivos Específicos:	3
2. Creacion de la carpeta app.....	3
2.1. Creación de los Controller	3
2.2. Creación de la carpeta Core	4
2.2.1. Creación de archivo Router.php	4
2.3. Creación de la carpeta models	5
2.3.1. Modelo de Autor.....	5
2.3.2. Modelo de Libro.....	5
2.4. Creación de los Repositorios	6
2.4.1. Repositorio de Autor.....	6
2.4.2. Repositorio de Libro.....	6
2.5. Creación de los Services	7
3. Creación de los archivos de la carpeta config.....	8
3.1. Creacion del bd.sql	8
3.2. Creación del Database	9
4. Creación de la carpeta public.....	9
4.1. Creación de los archivos JS	9
4.2. Uso de Axios para las Peticiones	13
4.3. Creación de los Templeates.....	14
4.3.1. Archivo footer	14
4.3.2. Archivo header.....	15
4.3.3. Creación del archivo .htaccess.....	15
4.3.4. Creación de los archivos autores.php.....	16
4.3.5. Creación de los archivos libros.php	17
4.3.6. Código del archivo index.php	19
4.3.7. Código del archivo inicio.php.....	20
Resultados.....	21
Conclusiones	21
Recomendaciones.....	21

1. Objetivos

1.1. Objetivo General

- ❖ Desarrollar una aplicación web funcional para la gestión de libros y autores aplicando el modelo MVC en PHP y permita realizar las opciones de crear, editar y eliminar algún dato de las listas.

1.2. Objetivos Específicos:

- ❖ Diseñar la aplicación web en base a los conocimientos adquiridos durante el periodo de clases.
- ❖ Establecer los elementos requeridos para el desarrollo de la aplicación web implementando peticiones con Axios, Bootstrap, modales y menú de navegación para una navegación más accesible.
- ❖ Trabajar en el desarrollo de la actividad de forma grupal y para obtener mejores resultados se sugiere asignar a cada integrante una estructura específica.

2. Creación de la carpeta app

2.1. Creación de los Controller

Creamos los archivos Controller “Autor y Libro PHP” nos permite recoger las peticiones y las trasfiere al modelo. Además, utilizamos la conexión con la base de datos utilizando el método JSON devolviendo una respuesta fácil de leer y comprender.

```
AutorController.php X
actividad3 > app > controllers > AutorController.php
1  <?php
2  require_once __DIR__ . '/../../config/database.php';
3  require_once __DIR__ . '/../services/AutorService.php';
4
5
6  class AutorController
7  {
8      private $autorService;
9
10     public function __construct()
11     {
12         // Establece la conexión con la base de datos y crea una instancia del servicio de autores
13         $database = new Database();
14         $db = $database->getConnection();
15         $this->autorService = new AutorService($db);
16     }
17
18     // Obtiene y devuelve todos los autores en formato JSON
19     public function index()
20     {
21         $result = $this->autorService->getAll();
22         echo json_encode($result);
23     }
24
25     /*Busca un autor por su ID y devuelve sus datos en formato JSON
26     y si el autor no se encuentra, devuelve un error 404.*/
27     public function show($id)
28     {
29         $result = $this->autorService->getById($id);
30         if ($result) {
31             echo json_encode($result);
32         } else {
33             http_response_code(404);
34             echo json_encode(['message' => 'No se encontro el autor']);
35         }
36     }
37 }
```

```
LibroController.php X
actividad3 > app > controllers > LibroController.php
1 <?php
2 require_once __DIR__ . '/../services/LibroService.php';
3 require_once __DIR__ . '/../config/database.php';
4
5 class LibroController
6 {
7     private $libroService;
8
9     public function __construct()
10    {
11        // Establece la conexión con la base de datos y crea una instancia del servicio de libros
12        $database = new Database();
13        $db = $database->getConnection();
14        $this->libroService = new LibroService($db);
15    }
16
17    // Obtiene y devuelve todos los libros en formato JSON
18    public function index()
19    {
20        $result = $this->libroService->getAll();
21        echo json_encode($result);
22    }
23
24    /*Busca un libro por su ID y devuelve sus datos en formato JSON
25    y si el libro no se encuentra, devuelve un error 404.*/
26    public function show($id)
27    {
28        $result = $this->libroService->getId($id);
29        if ($result) {
30            echo json_encode($result);
31        } else {
32            http_response_code(404);
33            echo json_encode(['message' => 'No se encontro el libro']);
34        }
35    }
36 }
37
```

2.2. Creación de la carpeta Core

2.2.1. Creación de archivo Router.php

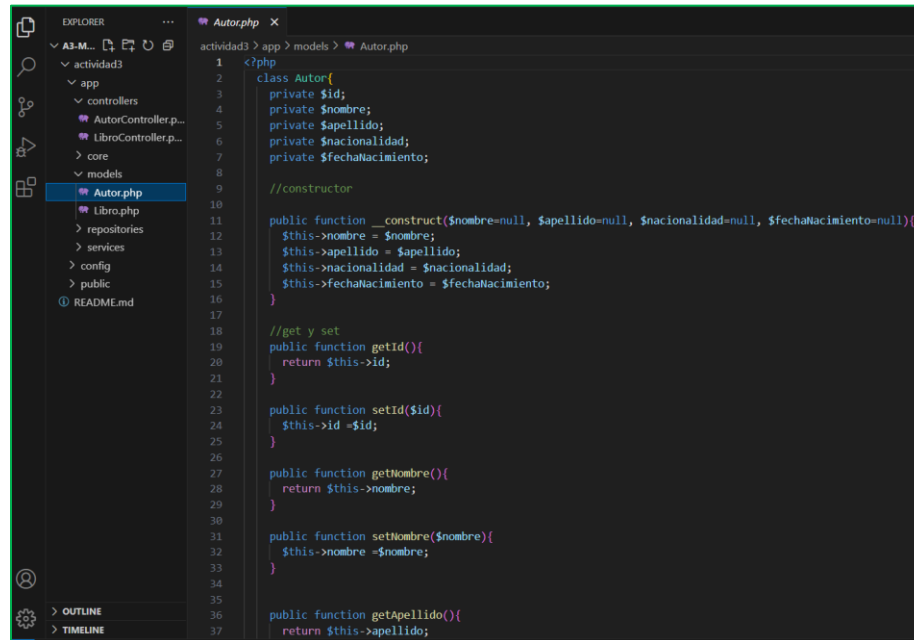
Elaboramos el código para nuestra clase “class Route “es la que se encarga de dirigir peticiones a rutas válidas para mapear las rutas de navegación a funciones o callbacks.

```
actividad3 > app > core > Router.php
1 <?php
2 /**
3  * Clase Router: Permite mapear rutas a funciones o callbacks.
4  * Se encarga de despachar la petición según el método HTTP y la URI.
5  */
6 class Router {
7     private $routes = [];
8
9     public function add($method, $route, $callback) {
10        $this->routes[] = [
11            'method' => strtolower($method),
12            'route' => $route,
13            'callback' => $callback
14        ];
15    }
16
17    public function dispatch($method, $uri) {
18        foreach ($this->routes as $route) {
19            if ($route['method'] === strtolower($method) && preg_match($this->convertRoute($route['route'], $uri), $uri)) {
20                array_shift($route['params']);
21                return call_user_func_array($route['callback'], $route['params']);
22            }
23        }
24        http_response_code(404);
25        echo json_encode(['message' => 'Not Found']);
26    }
27
28    private function convertRoute($route) {
29        return "#^" . preg_replace('/\\\\\\\[a-zA-Z0-9_]+/', '([a-zA-Z0-9_]+)', preg_quote($route)) . "$#";
30    }
31 }
32 ?>
```

2.3. Creación de la carpeta models

2.3.1. Modelo de Autor

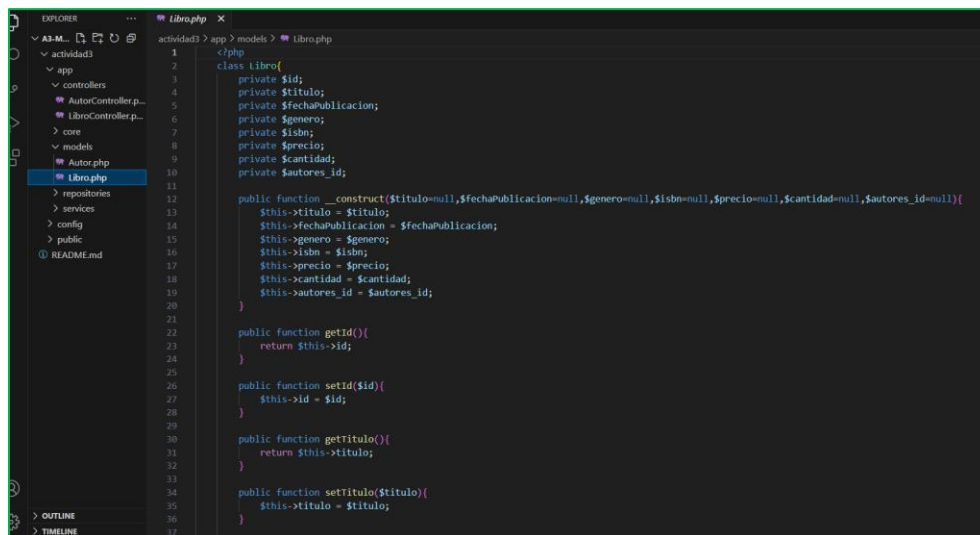
Creamos una clase “class Autor” y creamos los atributos del objeto Autor con sus respectivos getters and setters.



```
1 <?php
2 class Autor{
3     private $id;
4     private $nombre;
5     private $apellido;
6     private $nacionalidad;
7     private $fechaNacimiento;
8
9     //constructor
10
11     public function __construct($nombre=null, $apellido=null, $nacionalidad=null, $fechaNacimiento=null){
12         $this->nombre = $nombre;
13         $this->apellido = $apellido;
14         $this->nacionalidad = $nacionalidad;
15         $this->fechaNacimiento = $fechaNacimiento;
16     }
17
18     //get y set
19     public function getId(){
20         return $this->id;
21     }
22
23     public function setId($id){
24         $this->id = $id;
25     }
26
27     public function getNombre(){
28         return $this->nombre;
29     }
30
31     public function setNombre($nombre){
32         $this->nombre = $nombre;
33     }
34
35
36     public function getApellido(){
37         return $this->apellido;
```

2.3.2. Modelo de Libro

Creamos una clase “class Libro” y creamos los atributos del objeto Libro con sus respectivos getters y setters



```
1 <?php
2 class Libro{
3     private $id;
4     private $titulo;
5     private $fechaPublicacion;
6     private $genero;
7     private $isbn;
8     private $precio;
9     private $cantidad;
10     private $autores_id;
11
12     public function __construct($titulo=null, $fechaPublicacion=null, $genero=null, $isbn=null, $precio=null, $cantidad=null, $autores_id=null){
13         $this->titulo = $titulo;
14         $this->fechaPublicacion = $fechaPublicacion;
15         $this->genero = $genero;
16         $this->isbn = $isbn;
17         $this->precio = $precio;
18         $this->cantidad = $cantidad;
19         $this->autores_id = $autores_id;
20     }
21
22     public function getId(){
23         return $this->id;
24     }
25
26     public function setId($id){
27         $this->id = $id;
28     }
29
30     public function getTitulo(){
31         return $this->titulo;
32     }
33
34     public function setTitulo($titulo){
35         $this->titulo = $titulo;
36     }
37
38     public function setAutores($autores_id){
39         $this->autores_id = $autores_id;
40     }
41 }
```

2.4. Creación de los Repositorios

2.4.1. Repositorio de Autor

Creamos una clase “class AutorRepository” y creamos las funciones necesarias para las operaciones CRUD (obtener, crear, editar, eliminar) de un autor.

```
AutorRepository.php
actividad3 > app > repositories > AutorRepository.php
1 <?php
2 require_once __DIR__ . '/../models/Autor.php';
3
4 class AutorRepository{
5     private $conn;
6     private $table_name="autores";
7
8     //Recibe la conexión a la base de datos y lo almacena
9     public function __construct($db){
10         $this->conn=$db;
11     }
12
13     //Funcion que permite la inserción de un nuevo autor en la base de datos
14     public function create(Autor $autor){
15         $query = "INSERT INTO ($this->table_name) (nombre,apellido,nacionalidad,fechaNacimiento) VALUES (:nombre, :apellido, :nacionalidad, :fechaNa";
16         $stmt = $this->conn->prepare($query);
17         $stmt->bindParam(":nombre", $autor->getNombre());
18         $stmt->bindParam(":apellido", $autor->getApellido());
19         $stmt->bindParam(":nacionalidad", $autor->getNacionalidad());
20         $stmt->bindParam(":fechaNacimiento", $autor->getFechaNacimiento());
21         return $stmt->execute();
22     }
23
24     //Funcion que recupera todos los registros de la tabla
25     public function readAll(){
26         $query = "SELECT * FROM ($this->table_name)";
27         $stmt = $this->conn->prepare($query);
28         $stmt->execute();
29         return $stmt;
30     }
31
32     //Funcion que actualiza los datos de un autor existente
33     public function update(Autor $autor){
34         $query = "UPDATE ($this->table_name) SET nombre = :nombre, apellido = :apellido, nacionalidad = :nacionalidad, fechaNacimiento = :fechaNacimie";
35         $stmt = $this->conn->prepare($query);
36     }
```

2.4.2. Repositorio de Libro

Creamos una clase “class LibroRepository” y creamos las funciones necesarias para las operaciones CRUD (obtener, crear, editar, eliminar) de un libro.

```
LibroRepository.php
actividad3 > app > repositories > LibroRepository.php
1 <?php
2 require_once __DIR__ . '/../models/Libro.php';
3
4 class LibroRepository{
5     private $conn;
6     private $table_name = "libros";
7
8     //Recibe la conexión a la base de datos y lo almacena
9     public function __construct($db){
10         $this->conn = $db;
11     }
12
13     //Funcion que recupera todos los registros de la tabla
14     public function readAll(){
15         $query = "SELECT l.*, a.nombre as autor_nombre";
16         $query .= "FROM ($this->table_name) l";
17         $query .= "INNER JOIN autores a ON l.autores_id = a.id";
18         $stmt = $this->conn->prepare($query);
19         $stmt->execute();
20         return $stmt;
21     }
22
23     //Funcion que permite la inserción de un nuevo libro en la base de datos
24     public function create(Libro $libro){
25         $query = "INSERT INTO ($this->table_name) (titulo,fechaPublicacion,genero,isbn,precio,cantidad,autores_id)";
26         $query .= "VALUES (:titulo,:fechaPublicacion,:genero,:isbn,:precio,:cantidad,:autores_id)";
27         $stmt = $this->conn->prepare($query);
28         $stmt->bindParam(':titulo', $libro->getTitulo());
29         $stmt->bindParam(':fechaPublicacion', $libro->getFechaPublicacion());
30         $stmt->bindParam(':genero', $libro->getGenero());
31         $stmt->bindParam(':isbn', $libro->getIsbn());
32         $stmt->bindParam(':precio', $libro->getPrecio());
33         $stmt->bindParam(':cantidad', $libro->getCantidad());
34         $stmt->bindParam(':autores_id', $libro->getAutoresId());
35         return $stmt->execute();
36     }
37 }
```

2.5. Creación de los Services

Estos servicios en general actúan como intermediario entre el controlador y el repositorio

AutorService

```
1 // AutorService.php
2
3 // namespace
4 namespace App\Services;
5
6 // require_once
7 require_once __DIR__ . '/../repositories/autorRepository.php';
8
9 // class
10 class AutorService {
11     private $autorRepository;
12
13     public function __construct($db){
14         $this->autorRepository = new AutorRepository($db);
15     }
16
17     //función que obtiene todos los autores de la base de datos y los devuelve como un arreglo asociativo.
18     public function getAll(){
19         $total = $this->autorRepository->readAll();
20         $result = [];
21         while ($row = $total->fetch(PDO::FETCH_ASSOC)) {
22             $result[] = $row;
23         }
24         return $result;
25     }
26
27     //esta función permite obtener un autor por su ID y lo devuelve como contrario si no existe retorna null
28     public function getById($id){
29         $data = $this->autorRepository->read($id);
30         return $data ? $data : null;
31     }
32
33     //Crea un nuevo autor
34     public function create($data){
35         $autor = new Autor(); //Crea una nueva instancia de la clase
36         $autor->setNombre($data->nombre); //Establece el nombre del autor en data
37         $autor->setApellido($data->apellido); //Establece el apellido del autor en data
38         $autor->setNacionalidad($data->nacionalidad); //Asigna la nacionalidad del autor con el valor data
39         $autor->setFechaNacimiento($data->fechaNacimiento); //Guarda la fecha de nacimiento del autor con el valor data
40         return $this->autorRepository->create($autor); //Llama al método create del repositorio para crear el autor
41     }
42
43     //Función que actualiza los datos del autor (del mismo modo en cada línea hace lo mismo para actualizar)
44     public function update($data){
45         $autor = new Autor();
46         $autor->setId($data->id);
47         $autor->setNombre($data->nombre);
48         $autor->setApellido($data->apellido);
49         $autor->setNacionalidad($data->nacionalidad);
50         $autor->setFechaNacimiento($data->fechaNacimiento);
51         return $this->autorRepository->update($autor); //Llama al método update del repositorio para actualizar la información del autor
52     }
53
54     //Elimina un autor de la base de datos según su ID.
55     public function delete($id){
56         return $this->autorRepository->delete($id); //Llama al método delete del repositorio para eliminar el autor
57     }
58 }
```

```
1 //Crea un nuevo autor
2 public function create($data){
3     $autor = new Autor(); //Crea una nueva instancia de la clase
4     $autor->setNombre($data->nombre); //Establece el nombre del autor en data
5     $autor->setApellido($data->apellido); //Establece el apellido del autor en data
6     $autor->setNacionalidad($data->nacionalidad); //Asigna la nacionalidad del autor con el valor data
7     $autor->setFechaNacimiento($data->fechaNacimiento); //Guarda la fecha de nacimiento del autor con el valor data
8     return $this->autorRepository->create($autor); //Llama al método create del repositorio para crear el autor
9 }
10
11 //Función que actualiza los datos del autor (del mismo modo en cada línea hace lo mismo para actualizar)
12 public function update($data){
13     $autor = new Autor();
14     $autor->setId($data->id);
15     $autor->setNombre($data->nombre);
16     $autor->setApellido($data->apellido);
17     $autor->setNacionalidad($data->nacionalidad);
18     $autor->setFechaNacimiento($data->fechaNacimiento);
19     return $this->autorRepository->update($autor); //Llama al método update del repositorio para actualizar la información del autor
20 }
21
22 //Elimina un autor de la base de datos según su ID.
23 public function delete($id){
24     return $this->autorRepository->delete($id); //Llama al método delete del repositorio para eliminar el autor
25 }
26 }
```

LibroService

```
1 // LibroService.php
2
3 // namespace
4 namespace App\Services;
5
6 // require_once
7 require_once __DIR__ . '/../models/libro.php';
8 require_once __DIR__ . '/../repositories/libroRepository.php';
9
10 // class
11 class LibroService {
12     private $libroRepository;
13
14     public function __construct($db){
15         $this->libroRepository = new LibroRepository($db);
16     }
17
18     //Función que obtiene todos los libros de la base de datos y los devuelve como un arreglo asociativo.
19     public function getAll(){
20         $total = $this->libroRepository->readAll();
21         $result = [];
22         while ($row = $total->fetch(PDO::FETCH_ASSOC)) {
23             $result[] = $row;
24         }
25         return $result;
26     }
27
28     //esta función permite obtener un libro por su ID y lo devuelve como contrario si no existe retorna null
29     public function getById($id){
30         $data = $this->libroRepository->read($id);
31         return $data ? $data : null;
32     }
33
34     //Crea un nuevo libro
35     public function create($data){
36         $libro = new Libro(); //Crea una nueva instancia de la clase
37         $libro->setTitulo($data->titulo); //Establece el título del libro en data
38         $libro->setFechaPublicacion($data->fechaPublicacion); //Establece la fecha de publicación del libro en data
39         $libro->setGenero($data->genero); //Establece el género del libro en data
40         $libro->setIsbn($data->isbn); //Establece el ISBN del libro en data
41         $libro->setPrecio($data->precio); //Se asigna el precio del libro en data
42         $libro->setCantidad($data->cantidad); //Se asigna la cantidad de libros en data
43         $libro->setAutoresId($data->autores_id); //Se asigna el respectivo autor
44         return $this->libroRepository->create($libro); //Llama al método create del repositorio para crear el libro
45     }
46
47     //Función que actualiza los datos del libro (del mismo modo en cada línea hace lo mismo para actualizar)
48     public function update($data){
49         $libro = new Libro();
50         $libro->setId($data->id);
51         $libro->setTitulo($data->titulo);
52         $libro->setFechaPublicacion($data->fechaPublicacion);
53         $libro->setGenero($data->genero);
54         $libro->setIsbn($data->isbn);
55         $libro->setPrecio($data->precio);
56         $libro->setCantidad($data->cantidad);
57         $libro->setAutoresId($data->autores_id);
58         return $this->libroRepository->update($libro); //Llama al método update del repositorio para actualizar la información del libro
59     }
60
61     //Elimina un libro de la base de datos según su ID.
62     public function delete($id){
63         return $this->libroRepository->delete($id); //Llama al método delete del repositorio para eliminar el libro
64     }
65 }
```

```
1 $libro->setIsbn($data->isbn); //Establece el isbn del libro en data
2 $libro->setPrecio($data->precio); //Se asigna el precio del libro en data
3 $libro->setCantidad($data->cantidad); //Se asigna la cantidad de libros en data
4 $libro->setAutoresId($data->autores_id); //Se asigna el respectivo autor
5 return $this->libroRepository->create($libro); //Llama al método create del repositorio para crear el libro
6 }
7
8 //Función que actualiza los datos del libro (del mismo modo en cada línea hace lo mismo para actualizar)
9 public function update($data){
10     $libro = new Libro();
11     $libro->setId($data->id);
12     $libro->setTitulo($data->titulo);
13     $libro->setFechaPublicacion($data->fechaPublicacion);
14     $libro->setGenero($data->genero);
15     $libro->setIsbn($data->isbn);
16     $libro->setPrecio($data->precio);
17     $libro->setCantidad($data->cantidad);
18     $libro->setAutoresId($data->autores_id);
19     return $this->libroRepository->update($libro); //Llama al método update del repositorio para actualizar la información del libro
20 }
21
22 //Elimina un libro de la base de datos según su ID.
23 public function delete($id){
24     return $this->libroRepository->delete($id); //Llama al método delete del repositorio para eliminar el libro
25 }
26 }
```

3. Creación de los archivos de la carpeta config

3.1. Creacion del bd.sql

Este archivo, contiene el script que se insertó en MySQL para la creación de la base de datos y adicional contiene 5 inserciones para cada tabla.

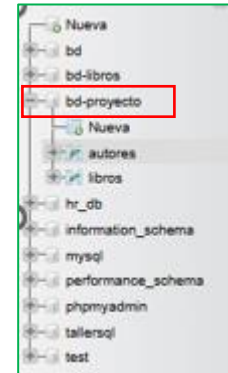
```
A3-MVC-php--main > actividad3 > config > bd.sql
1 CREATE TABLE autores (
2     id INT AUTO_INCREMENT PRIMARY KEY,
3     nombre VARCHAR(100) NOT NULL,
4     apellido VARCHAR(100) NOT NULL,
5     nacionalidad VARCHAR(100) NOT NULL,
6     fechaNacimiento DATE NOT NULL
7 );
8
9 CREATE TABLE libros (
10     id INT AUTO_INCREMENT PRIMARY KEY,
11     titulo VARCHAR(255) NOT NULL,
12     fechaPublicacion DATE NOT NULL,
13     genero VARCHAR(100) NOT NULL,
14     isbn VARCHAR(20) UNIQUE NOT NULL,
15     precio DECIMAL(10,2) NOT NULL,
16     cantidad INT NOT NULL,
17     autores_id INT NOT NULL,
18     FOREIGN KEY (autores_id) REFERENCES autores(id) ON DELETE CASCADE
19 );
20
21 INSERT INTO autores (nombre, apellido, nacionalidad, fechaNacimiento)
22 VALUES
23     ('Gabriel', 'García Márquez', 'Colombiana', '1927-03-06'),
24     ('J.K.', 'Rowling', 'Británica', '1965-07-31'),
25     ('Haruki', 'Murakami', 'Japonesa', '1949-01-12'),
26     ('Isabel', 'Allende', 'Chilena', '1942-08-02'),
27     ('Friedrich', 'Nietzsche', 'Alemana', '1844-10-15');
28
29
30 INSERT INTO libros (titulo, fechaPublicacion, genero, isbn, precio, cantidad, autores_id)
31 VALUES
32     ('Cien Años de Soledad', '1967-06-05', 'Realismo Mágico', '9780307474728', 15.99, 100, 1),
33     ('Harry Potter y la Piedra Filosofal', '1997-06-26', 'Fantasía', '978074532699', 12.99, 150, 2),
34     ('Kafka en la Orilla', '2002-09-12', 'Ficción Contemporánea', '9780307454750', 14.50, 120, 3),
35     ('La Casa de los Espíritus', '1982-05-05', 'Realismo Mágico', '9780345806892', 16.99, 80, 4),
36     ('Así Habló Zaratustra', '1883-12-25', 'Filosofía', '9780140441185', 9.99, 50, 5);
37
```

	id	nombre	apellido	nacionalidad	fechaNacimiento
☐	1	Gabriel	García Márquez	Colombiana	1927-03-06
☐	2	J.K.	Rowling	Británica	1965-07-31
☐	3	Haruki	Murakami	Japonesa	1949-01-12
☐	4	Isabel	Allende	Chilena	1942-08-02
☐	23	Franz	Kafka	Checo	1883-07-03

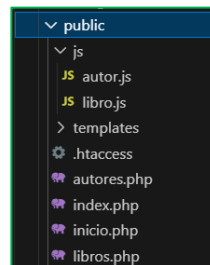
3.2. Creación del Database

Creamos una clase “class Database” que permite la conexión de a la base de datos encapsulando los detalles de la conexión como el host, nombre de la base de datos, en este caso “bd-proyecto”, usuario y contraseña, donde proporciona el método getConnection() para obtener una instancia de la conexión.

```
database.php X
actividad1 > config > database.php
1 <?php
2
3 class Database
4 {
5     private $host = "localhost";
6     private $db_name = "bd-proyecto"; // Definimos el nombre de la base de datos
7     private $username = "root";
8     private $password = "";
9     public $conn; // Variable para almacenar la conexión a la base de datos
10
11 //Método para obtener la conexión a la base de datos
12 public function getConnection()
13 {
14     // Crear una nueva conexión PDO a la base de datos
15     $this->conn = null;
16     try {
17         $this->conn = new PDO(
18             "mysql:host=" . $this->host . ";dbname=" . $this->db_name,
19             $this->username,
20             $this->password
21         );
22         $this->conn->exec("set names utf8");
23     } catch (PDOException $exception) {
24
25     }
26     // Muestra un mensaje de error con la descripción del problema
27     echo "Error de conexión: " . $exception->getMessage();
28 }
29
30 return $this->conn;
31
32 }
```



4. Creación de la carpeta public



Creamos una carpeta public que almacenará archivos JS, templates y htaccess, mejorando el orden, la reutilización de código y escalabilidad del proyecto

4.1. Creación de los archivos JS

En este apartado el código contiene las funciones para obtener datos y enviar al servidor y permite realizar las modificaciones requeridas por el usuario a través de URLs API.

Código JS para “Gestión de autores”

```
JS autorjs X
actividad3 > public > js > JS autorjs > ...
1 //Esta es la ruta base de la API, puede ser modificado segun el entorno de cada persona
2 const apiUrl='http://localhost/php-mvc-gl-ga/A3-MVC-php-/actividad3/public';
3
4 // Al cargar la página, se ejecuta la función para obtener todos los autores.
5 document.addEventListener('DOMContentLoaded', () => getAutores());
6
7 //Funcion que obtiene todos los autores desde la API y actualiza la tabla.
8 const getAutores = ()=> {
9   axios.get(`${apiUrl}/autores`)
10  .then(response => {
11    const autores = response.data; //Variable que guardar los datos enviados por el servidor
12    const tbody = document.querySelector("#autoresTable tbody"); //Variable que guarda la referencia del tbody para luego ser manipulado para la o
13    tbody.innerHTML = '';
14    //forEach que recorre el array de autores y crea dinamicamente las filas tr
15    autores.forEach(author => {
16      const tr = document.createElement('tr');
17      tr.innerHTML = `
18        <td>${author.id}</td>
19        <td>${author.nombre}</td>
20        <td>${author.apellido}</td>
21        <td>${author.nacionalidad}</td>
22        <td>${author.fechaNacimiento}</td>
23        <td>
24          <button class="btn btn-sm btn-info" onclick="editAutor(${author.id})">Editar</button>
25          <button class="btn btn-sm btn-danger" onclick="deleteAutor(${author.id})">Eliminar</button>
26        </td>
27      `;
28      tbody.appendChild(tr); //Se encarga de agregar un elemnto de fila a la tabla de autores
29    });
30  })
31  .catch(error => console.error(error));
32 };
33
34 //Abre el modal para agregar un nuevo autor.
35 const openModalAutor = () => {
36   document.getElementById('autorForm').reset();
37   document.getElementById('autorId').value = '';
```

```
JS autorjs X
actividad3 > public > js > JS autorjs > [getAutores] then() callback
35 const openModalAutor = () => {
36
37
38
39 };
40
41 //Envía el formulario para crear o actualizar un autor, gracia el evento del boton guardar.
42 document.getElementById('autorForm').addEventListener('submit', e => {
43   e.preventDefault();
44
45   //Se obtiene el valor de cada campo del formulario y se almacena en variables
46   const id = document.getElementById('autorId').value;
47   const nombre = document.getElementById('autorNombre').value;
48   const apellido = document.getElementById('autorApellido').value;
49   const nacionalidad = document.getElementById('autorNacionalidad').value;
50   const fechaNacimiento = document.getElementById('autorFechaNacimiento').value;
51
52   //Condional if que verifica si el 'id' existe, es decir el autor ya está en la base de datos por ende sera una actualizacion
53   if (id) {
54
55     //Hace la solicitud PUT
56     axios.put(`${apiUrl}/autores`, { id, nombre, apellido, nacionalidad, fechaNacimiento })
57     .then(response => {
58
59       alert("Autor actualizado correctamente"); //Mensaje de que la ejecucion fue correcta
60       $('#autorModal').modal('hide');
61       getAutores();
62     })
63     .catch(error => console.error(error));
64   } else {
65
66     //Si no existe el id entonces creara el autor con la solicitud POST
67     axios.post(`${apiUrl}/autores`, { nombre, apellido, nacionalidad, fechaNacimiento })
68     .then(response => {
69
70       alert("Autor agregado correctamente"); //Mensaje de que la ejecucion fue correcta
71       $('#autorModal').modal('hide');
72       getAutores();
73     })
74     .catch(error => console.error(error));
```

```
JS autorjs
actividad3 > public > js > JS autorjs > ...
42 document.getElementById('autorForm').addEventListener('submit', e => {
74     .catch(error => console.error(error));
75 }
76 });
77
78 //Carga los datos de un autor en el formulario para editar.
79 const editAutor = id => {
80
81     //Hace solicitud GET para obtener los datos del autor
82     axios.get(`${apiUrl}/autores/${id}`)
83         .then(response => {
84             const autor = response.data;
85             document.getElementById('autorId').value = autor.id;
86             document.getElementById('autorNombre').value = autor.nombre;
87             document.getElementById('autorApellido').value = autor.apellido;
88             document.getElementById('autorNacionalidad').value = autor.nacionalidad;
89             document.getElementById('autorFechaNacimiento').value = autor.fechaNacimiento;
90             document.getElementById('autorModallabel').innerText = 'Editar Autor';
91             $('#autorModal').modal('show');
92         })
93         .catch(error => console.error(error));
94     };
95
96     //Elimina un autor.
97     //
98     //Función para eliminar un autor que recibe el 'id' del autor como parámetro
99     const deleteAutor = id => {
100         if (confirm('¿Estás seguro de eliminar este autor?')) {
101             axios.delete(`${apiUrl}/autores`, { data: { id } })
102                 .then(response => {
103                     alert("Autor eliminado correctamente");
104                     getAutores();
105                 })
106                 .catch(error => console.error(error));
107         }
108     };
109 }
```

Código JS para “Gestión de Libros”

```
actividad3 > public > js > JS libros.js > getLibros > then() callback > libros.forEach() callback
1 //Esta es la ruta base de la API, puede ser modificado segun el entorno de cada persona
2 const apiUrl = 'http://localhost/php-mvc-g1-ga/A3-MVC-php-/actividad3/public';
3
4 // Al cargar la página, se ejecuta la función para obtener todos los libros.
5 document.addEventListener('DOMContentLoaded', () => getLibros());
6
7 //Funcion que obtiene todos los libros desde la API y actualiza la tabla.
8 const getLibros = () => {
9     axios.get(`${apiUrl}/libros`)
10         .then(response => {
11             const libros = response.data; //Variable que guardar los datos enviados por el servidor
12             const tbody = document.querySelector('#librosTable tbody'); //Variable que guarda la referencia del tbody para luego ser manipulado para la ob
13             tbody.innerHTML = '';
14             //forEach que recorre el array de libros y crea dinamicamente las filas tr
15             libros.forEach(libro => {
16                 const tr = document.createElement('tr');
17                 tr.innerHTML = `
18                     <td>${libro.id}</td>
19                     <td>${libro.titulo}</td>
20                     <td>${libro.autor_nombre}</td>
21                     <td>${libro.fechaPublicacion}</td>
22                     <td>${libro.genero}</td>
23                     <td>${libro.isbn}</td>
24                     <td>${libro.precio}</td>
25                     <td>${libro.cantidad}</td>
26                     <td>
27                         <button class="btn btn-sm btn-info" onclick="editLibro(${libro.id})">Editar</button>
28                         <button class="btn btn-sm btn-danger" onclick="deletelibro(${libro.id})">Eliminar</button>
29                     </td>
30                 `;
31                 tbody.appendChild(tr); //Se encarga de agregar un elemnto de fila a la tabla de libros
32             });
33         })
34         .catch(error => console.error(error));
35     };
36
37 }
```

```

47 document.getElementById('libroForm').addEventListener('submit', e => {
73
74     //Hace la solicitud PUT
75     axios.put(`${apiUrl}/libros`, { id, titulo, autores_id, fechaPublicacion, genero, isbn, precio, cantidad })
76     .then(response => {
77
78         alert('Libro actualizado correctamente'); //Mensaje de que la ejecucion fue correcta
79         $('#libroModal').modal('hide');
80         getLibros();
81     })
82     .catch(error => console.error(error));
83 } else {
84
85     //Si no existe el id entonces creara el libro con la solicitud POST
86     axios.post(`${apiUrl}/libros`, { titulo, autores_id, fechaPublicacion, genero, isbn, precio, cantidad })
87     .then(response => {
88         alert('Libro agregado correctamente'); //Mensaje de que la ejecucion fue correcta
89         $('#libroModal').modal('hide');
90         getLibros();
91     })
92     .catch(error => console.error(error));
93 }
94 });
95
96
97 //Carga los datos de un libro en el formulario para editar.
98
99 const editLibro = id => {
100
101     //Hace solicitud GET para obtener los datos del libro
102     axios.get(`${apiUrl}/libros/${id}`)
103     .then(response => {
104         const libro = response.data;
105         document.getElementById('libroId').value = libro.id;
106         document.getElementById('libroTitulo').value = libro.titulo;
107         document.getElementById('libroAutor').value = libro.autores_id;
108         document.getElementById('libroFechaPublicacion').value = libro.fechaPublicacion;

```

```

actividad3 > public > js > libro.js > getLibros > then() callback > libros.forEach() callback
38 //Abre el modal para agregar un nuevo libro.
39 const openModalLibro = () => {
actividad3 > public > js > libro.js > getLibros > then() callback > libros.forEach() callback
117
118
119 //Elimina un libro.
120
121 // Función para eliminar un libro que recibe el 'id' del libro como parámetro
122 const deleteLibro = id => {
123
124     //Se lanza una tipo alerta si quiere eliminar el libro
125     if (confirm('¿Estás seguro de eliminar este libro?')) {
126
127         //Si confirma lo eliminara con la solicitud DELETE
128         axios.delete(`${apiUrl}/libros`, { data: { id } })
129         .then(response => {
130             alert('Libro eliminado correctamente'); // Alerta después de eliminar el libro
131             getLibros(); // Actualiza la lista de libros luego de la accion
132         })
133         .catch(error => console.error(error));
134     }
135 };
136
137 //Obtiene todos los autores y llena el select que contiene los autores creados en el formulario de libros.
138 const llenarSelectAutores = () => {
139     axios.get(`${apiUrl}/autores`)
140     .then(response => {
141         const autores = response.data;
142         const selectAutor = document.getElementById("libroAutor");
143
144         // Limpiar el select antes de agregar nuevas opciones
145         selectAutor.innerHTML = "";
146
147         // Agregar una opción por defecto
148         let optionDefault = document.createElement("option");
149         optionDefault.value = "";
150         optionDefault.textContent = "Seleccione un autor"; //Opcion defecto
151         optionDefault.disabled = true;
152         optionDefault.selected = true;
153         selectAutor.appendChild(optionDefault);

```



```

3      selectAutor.appendChild(optionDefault);
4
5      // Recorrer los autores y agregarlos al select
6      autores.forEach(autor => {
7          let option = document.createElement("option");
8          option.value = autor.id;
9          option.textContent = `${autor.nombre} ${autor.apellido}`;
10         selectAutor.appendChild(option);
11     });
12 }
13 .catch(error => console.error(error));
14 };
15
16 // Llamar a la función cuando la página se cargue
17 document.addEventListener("DOMContentLoaded", () => {
18     llenarSelectAutores();
19     getLibros();
20 });

```

4.2. Uso de Axios para las Peticiones

Axios es una herramienta clave en el frontend para gestionar solicitudes http hacia la API, de esta forma facilita la interacción con el backend mediante “promesas”.

Agregar autor:

```

axios.put(`${apiUrl}/autores`, { id, nombre, apellido, nacionalidad, fechaNacimiento })
    .then(response => {
        alert("Autor actualizado correctamente"); //Mensaje de que la ejecucion fue correcta
        $('#autorModal').modal('hide');
        getAutores();
    })
    .catch(error => console.error(error));
} else {

```

Editar autor:

```

//Si no existe el id entonces creara el autor con la solicitud POST
axios.post(`${apiUrl}/autores`, { nombre, apellido, nacionalidad, fechaNacimiento })
    .then(response => {
        alert("Autor agregado correctamente"); //Mensaje de que la ejecucion fue correcta
        $('#autorModal').modal('hide');
        getAutores();
    })
    .catch(error => console.error(error));
}
});

```

Obtener autores:

```
//Hace solicitud GET para obtener los datos del autor
axios.get(`${apiUrl}/autores/${id}`)
  .then(response => {
    const autor = response.data;
    document.getElementById('autorId').value = autor.id;
    document.getElementById('autorNombre').value = autor.nombre;
    document.getElementById('autorApellido').value = autor.apellido;
    document.getElementById('autorNacionalidad').value = autor.nacionalidad;
    document.getElementById('autorFechaNacimiento').value = autor.fechaNacimiento;
    document.getElementById('autorModalLabel').innerText = 'Editar Autor';
    $('#autorModal').modal('show');
  })
  .catch(error => console.error(error));
};
```

Gestión de Autores

+ Agregar Autor

ID	Nombre	Apellido	Nacionalidad	Fecha de Nacimiento	Acciones
1	Gabriel	García Márquez	Colombiana	1927-03-06	<button>Editar</button> <button>Eliminar</button>
2	J.K.	Rowling	Británica	1965-07-31	<button>Editar</button> <button>Eliminar</button>
3	Haruki	Murakami	Japonesa	1949-01-12	<button>Editar</button> <button>Eliminar</button>
4	Isabel	Allende	Chilena	1942-08-02	<button>Editar</button> <button>Eliminar</button>
5	Friedrich	Nietzsche	Alemana	1844-10-15	<button>Editar</button> <button>Eliminar</button>

Eliminar autor:

```
//Función para eliminar un autor que recibe el 'id' del autor como parámetro
const deleteAutor = id => {
  if (confirm('¿Estás seguro de eliminar este autor?')) {
    axios.delete(`${apiUrl}/autores`, { data: { id } })
      .then(response => {
        alert("Autor eliminado correctamente");
        getAutores();
      })
      .catch(error => console.error(error));
  }
};
```

localhost dice

Autor eliminado correctamente

Aceptar

4.3. Creación de los Templates

4.3.1. Archivo footer

Es un componente que se encontrara visible para el usuario, aquí se integran los links de jQuery, Bootstrap para dar estilos y Axios para tratar la salida de datos en formato JSON.

```
JS autor.js  footer.php X
actividad3 > public > templates > footer.php
1 <!--FOOTER PARA PAGINAS-->
2 <footer class="bg-dark text-white text-center py-3 mt-auto">
3   <div class="container">
4     <p class="mb-0">© 2025 APLICACIÓN TECNOLOGÍAS WEB | Gestión de Libros y Autores</p>
5   </div>
6 </footer>
7
8 <!--SCRIPTS-->
9 <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
10 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
11 <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
12
13 </body>
14 </html>
15
16
```

4.3.2. Archivo header

El código header será el mismo en todas las páginas de navegación ya que permitirá visualizar las mismas opciones en todas las secciones de barra, para ello el código será el mismo en todas las páginas.

```
4 <!--header-->
5 <!--header-->
6 <!--header-->
7 <!--header-->
8 <!--header-->
9 <!--header-->
10 <!--header-->
11 <!--header-->
12 <!--header-->
13 <!--header-->
14 <!--header-->
15 <!--header-->
16 <!--header-->
17 <!--header-->
18 <!--header-->
19 <!--header-->
20 <!--header-->
21 <!--header-->
22 <!--header-->
23 <!--header-->
24 <!--header-->
25 <!--header-->
26 <!--header-->
27 <!--header-->
28 <!--header-->
29 <!--header-->
30 <!--header-->
31 <!--header-->
32 <!--header-->
33 <!--header-->
34 <!--header-->
35 <!--header-->
36 <!--header-->
37 <!--header-->
38 <!--header-->
39 <!--header-->
40 <!--header-->
41 <!--header-->
42 <!--header-->
43 <!--header-->
44 <!--header-->
45 <!--header-->
46 <!--header-->
47 <!--header-->
48 <!--header-->
49 <!--header-->
50 <!--header-->
51 <!--header-->
52 <!--header-->
53 <!--header-->
54 <!--header-->
55 <!--header-->
56 <!--header-->
57 <!--header-->
58 <!--header-->
59 <!--header-->
60 <!--header-->
61 <!--header-->
62 <!--header-->
63 <!--header-->
64 <!--header-->
65 <!--header-->
66 <!--header-->
67 <!--header-->
68 <!--header-->
69 <!--header-->
70 <!--header-->
71 <!--header-->
72 <!--header-->
73 <!--header-->
74 <!--header-->
75 <!--header-->
76 <!--header-->
77 <!--header-->
78 <!--header-->
79 <!--header-->
80 <!--header-->
81 <!--header-->
82 <!--header-->
83 <!--header-->
84 <!--header-->
85 <!--header-->
86 <!--header-->
87 <!--header-->
88 <!--header-->
89 <!--header-->
90 <!--header-->
91 <!--header-->
92 <!--header-->
93 <!--header-->
94 <!--header-->
95 <!--header-->
96 <!--header-->
97 <!--header-->
98 <!--header-->
99 <!--header-->
100 <!--header-->
```

4.3.3. Creación del archivo .htaccess

El archivo .htaccess permite redirigir todas las solicitudes a index.php, asegurando las URLs.

```
actividad3 > public > .htaccess
1 #Habilitar la reescritura de solicitudes de URL
2 RewriteEngine On
3
4 RewriteCond %{REQUEST_FILENAME} !-f
5 RewriteCond %{REQUEST_FILENAME} !-d
6
7 RewriteRule ^(.*)$ index.php [QSA,L]
8 |
```

4.3.4. Creación de los archivos autores.php

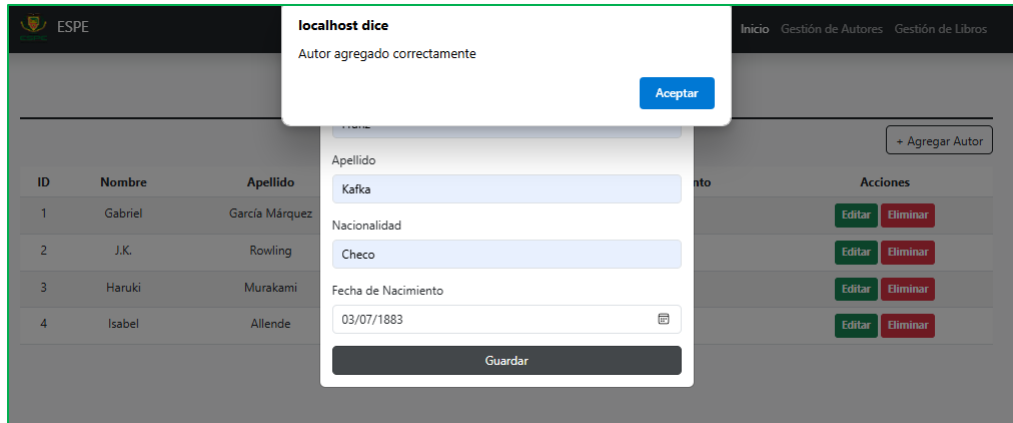
El archivo de autores.php gestiona los datos de autores con una tabla dinámica y un formulario modal, usando Bootstrap para el diseño y JavaScript (autor.js) para las acciones de CRUD. Además, incluye encabezado (header.php) y pie de página (footer.php).

```

1  <?php include 'templates/header.php'; ?> <!--SE INCLUYE EL HEADER QUE CONTIENE LOS NAVBAR-->
2
3  <!--DIV GENERAL QUE CONTIENE LA TABLA RENDERIZADA POR EL SCRIPT-->
4  <div class="container mt-5 pt-5">
5      <h1 class="text-center text-dark fw-bold pb-2 border-bottom border-3 border-dark">Gestión de Autores</h1>
6
7      <!--DIV QUE CONTIENE EL BOTON QUE HARA LA ACCION PARA MOSTRAR EL MODAL PARA EL REGISTRO DE AUTORES-->
8      <div class="d-flex justify-content-end mb-3">
9          <button class="btn btn-outline-dark" data-bs-toggle="modal" data-bs-target="#autorModal" onclick="openModalAutor();">
10              + Agregar Autor
11          </button>
12      </div>
13
14      <!--DIV QUE CONTIENE LA TABLA DE AUTORES-->
15      <div class="table-responsive">
16          <table id="autoresTable" class="table table-striped table-hover">
17              <thead class="bg-dark text-white text-center">
18                  <tr>
19                      <th>ID</th>
20                      <th>Nombre</th>
21                      <th>Apellido</th>
22                      <th>Nacionalidad</th>
23                      <th>Fecha de Nacimiento</th>
24                      <th>Acciones</th>
25                  </tr>
26              </thead>
27              <tbody class="text-center">
28                  <!-- Se llenarán dinámicamente mediante JavaScript -->
29              </tbody>
30          </table>
31      </div>
32  </div>
33
34  <!-- Modal para Autores por documentacion de bootstrap con los campos respectivos-->
35  <div class="modal fade" id="autorModal" tabindex="-1" aria-labelledby="autorModalLabel" aria-hidden="true">
36      <div class="modal-dialog">

```

```
<h5 id="autorModal" label="Agregar Autor">
<button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="Cerrar"></button>
</div>
<div class="modal-body">
<form id="autorForm">
<input type="hidden" id="autorId">
<div class="mb-3">
<label for="autorNombre" class="form-label">Nombre</label>
<input type="text" class="form-control" id="autorNombre" required>
</div>
<div class="mb-3">
<label for="autorApellido" class="form-label">Apellido</label>
<input type="text" class="form-control" id="autorApellido" required>
</div>
<div class="mb-3">
<label for="autorNacionalidad" class="form-label">Nacionalidad</label>
<input type="text" class="form-control" id="autorNacionalidad" required>
</div>
<div class="mb-3">
<label for="autorFechaNacimiento" class="form-label">Fecha de Nacimiento</label>
<input type="date" class="form-control" id="autorFechaNacimiento" required>
</div>
<div class="d-grid">
<button type="submit" class="btn btn-dark">Guardar</button>
</div>
</form>
</div>
</div>
</div>
<script src="js/autor.js"></script> <!--SCRIPT PARA LAS ACCIONES DE AGREGAR, EDITAR, ELIMINAR Y OBTENER-->
<?php include 'templates/footer.php'; ?> <!--SE INCLUYE EL FOOTER QUE CONTIENE EL DISEÑO Y LOS SCRIPTS NECESARIOS PARA ACCIONES DE MODALES, BOOTSTRAP-->
```



ID	Nombre	Apellido	Nacionalidad	Fecha de Nacimiento	Acciones
1	Gabriel	García Márquez	Colombiana	1927-03-06	<button>Editar</button> <button>Eliminar</button>
2	J.K.	Rowling	Británica	1965-07-31	<button>Editar</button> <button>Eliminar</button>
3	Haruki	Murakami	Japonesa	1949-01-12	<button>Editar</button> <button>Eliminar</button>
4	Isabel	Allende	Chilena	1942-08-02	<button>Editar</button> <button>Eliminar</button>
23	Franz	Kafka	Checo	1883-07-03	<button>Editar</button> <button>Eliminar</button>

4.3.5. Creación de los archivos libros.php

El código gestiona los libros ingresados en la aplicación, mostrando una tabla con sus datos y un botón para agregar nuevos registros mediante un modal. Usa header.php y footer.php para la estructura e incluye libro.js para las acciones CRUD.

```

1  <?php include 'templates/header.php'; ?> <!--SE INCLUYE EL HEADER QUE CONTIENE LOS NAVBAR-->
2
3  <!--DIV GENERAL QUE CONTIENE LA TABLA RENDERIZADA POR EL SCRIPT-->
4  <div class="container mt-5 pt-5">
5      <h1 class="text-center text-dark fw-bold pb-2 border-bottom border-3 border-dark">Gestión de Libros</h1>
6
7      <!--DIV QUE CONTIENE EL BOTON QUE HARA LA ACCION PARA MOSTRAR EL MODAL PARA EL REGISTRO DE LIBROS-->
8      <div class="d-flex justify-content-end mb-3">
9          <button class="btn btn-outline-dark" data-bs-toggle="modal" data-bs-target="#libroModal" onclick="openModalLibro();">
10              + Agregar Libro
11          </button>
12      </div>
13
14      <!--DIV QUE CONTIENE LA TABLA DE LIBROS-->
15      <div class="table-responsive">
16          <table id="librosTable" class="table table-striped table-hover">
17              <thead class="bg-dark text-white text-center">
18                  <tr>
19                      <th>ID</th>
20                      <th>Título</th>
21                      <th>Autor</th>
22                      <th>Año de publicación</th>
23                      <th>Género</th>
24                      <th>ISBN</th>
25                      <th>Precio</th>
26                      <th>Cantidad</th>
27                      <th>Acciones</th>
28                  </tr>
29              </thead>
30              <tbody class="text-center">
31                  <!-- Se llenarán dinámicamente mediante JavaScript -->
32              </tbody>
33          </table>
34      </div>
35  </div>
36

```



```

6
7 <!-- Modal para Libros por documentacion de bootstrap con los campos respectivos-->
8 <div class="modal fade" id="libroModal" tabindex="-1" aria-labelledby="libroModalLabel" aria-hidden="true">
9   <div class="modal-dialog">
10     <div class="modal-content">
11       <div class="modal-header bg-dark text-white">
12         <h5 id="libroModalLabel" class="modal-title">Agregar Libro</h5>
13         <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="Cerrar"></button>
14       </div>
15       <div class="modal-body">
16         <form id="libroForm">
17           <input type="hidden" id="libroId">
18           <div class="mb-3">
19             <label for="libroTitulo" class="form-label">Titulo</label>
20             <input type="text" class="form-control" id="libroTitulo" required>
21           </div>
22           <div class="mb-3">
23             <label for="libroAutor" class="form-label">Autor</label>
24             <select class="form-select" id="libroAutor" required>
25               <!-- Se llenará dinámicamente mediante JavaScript -->
26             </select>
27           </div>
28           <div class="mb-3">
29             <label for="libroFechaPublicacion" class="form-label">Fecha de publicación</label>
30             <input type="date" class="form-control" id="libroFechaPublicacion" required>
31           </div>
32           <div class="mb-3">
33             <label for="libroGenero" class="form-label">Género</label>
34             <input type="text" class="form-control" id="libroGenero" required>
35           </div>
36           <div class="mb-3">
37             <label for="libroIsbn" class="form-label">ISBN</label>
38             <input type="text" class="form-control" id="libroIsbn" required>
39           </div>
40           <div class="mb-3">
41             <label for="libroPrecio" class="form-label">Precio</label>

```

```

        <label for="libroFechaPublicacion" class="form-label">Fecha de publicación</label>
        <input type="date" class="form-control" id="libroFechaPublicacion" required>
      </div>
      <div class="mb-3">
        <label for="libroGenero" class="form-label">Género</label>
        <input type="text" class="form-control" id="libroGenero" required>
      </div>
      <div class="mb-3">
        <label for="libroIsbn" class="form-label">ISBN</label>
        <input type="text" class="form-control" id="libroIsbn" required>
      </div>
      <div class="mb-3">
        <label for="libroPrecio" class="form-label">Precio</label>
        <input type="number" class="form-control" id="libroPrecio" required>
      </div>
      <div class="mb-3">
        <label for="libroCantidad" class="form-label">Cantidad</label>
        <input type="number" class="form-control" id="libroCantidad" required>
      </div>
      <div class="d-grid">
        <button type="submit" class="btn btn-dark">Guardar</button>
      </div>
    </form>
  </div>
</div>
</div>

```

```

<script src="js/libro.js"></script> <!--SCRIPT PARA LAS ACCIONES DE AGREGAR, EDITAR, ELIMINAR Y OBTENER-->
<?php include 'templates/footer.php'; ?> <!--SE INCLUYE EL FOOTER QUE CONTIENE EL DISEÑO Y LOS SCRIPTS NECESARIOS PARA ACCIONES DE MODALES, BOOTSTRA

```

4.3.6. Código del archivo index.php

El script index.php define una API REST con rutas para gestionar autores y libros, usando un enrutador y controladores. También configura CORS y redirige solicitudes según la URL.

Gestión de Libros

+ Agregar Libro

ID	Título	Autor	Año de publicación	Género	ISBN	Precio	Cantidad	Acciones
1	Cien Años de Soledad	Gabriel	1967-06-05	Realismo Mágico	9780307474728	15.99	100	Editar Eliminar
2	Harry Potter y la Piedra Filosofal	J.K.	1997-06-26	Fantasia	9780747532699	12.99	150	Editar Eliminar
3	Kafka en la Orilla	Haruki	2002-09-12	Ficción Contemporánea	9780307454750	14.50	120	Editar Eliminar
4	La Casa de los Espíritus	Isabel	1982-05-05	Realismo Mágico	9780345806892	16.99	80	Editar Eliminar

localhost dice
Libro eliminado correctamente
[Aceptar](#)

Agregar Libro

Título
Autor: Gabriel García Márquez
Fecha de publicación: dd/mm/aaaa
Género
ISBN
Precio
Cantidad
[Guardar](#)

Editar Libro

Título: Cien Años de Soledad
Autor: Gabriel García Márquez
Fecha de publicación: 05/06/1967
Género: Realismo Mágico
ISBN: 9780307474728
Precio: 15.99
Cantidad: 100
[Guardar](#)

```
actividad3 > public > index.php
1 <?php
2 header("Content-Type: application/json; charset=UTF-8");
3 header("Access-Control-Allow-Origin: *");
4 header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");
5 header("Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization, X-Requested-With");
6
7 require_once __DIR__ . '/../app/core/Router.php';
8 require_once __DIR__ . '/../app/controllers/AutorController.php';
9 require_once __DIR__ . '/../app/controllers/LibroController.php';
10
11 $router = new Router();
12
13 $autorController = new AutorController();
14 $router->add('GET', '/autores', fn() => $autorController->index());
15 $router->add('GET', '/autores/:id', fn($id) => $autorController->show($id));
16 $router->add('POST', '/autores', fn() => $autorController->store());
17 $router->add('PUT', '/autores', fn() => $autorController->update());
18 $router->add('DELETE', '/autores', fn() => $autorController->destroy());
19
20 $libroController = new LibroController();
21 $router->add('GET', '/libros', fn() => $libroController->index());
22 $router->add('GET', '/libros/:id', fn($id) => $libroController->show($id));
23 $router->add('POST', '/libros', fn() => $libroController->store());
24 $router->add('PUT', '/libros', fn() => $libroController->update());
25 $router->add('DELETE', '/libros', fn() => $libroController->destroy());
26
27 $uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
28 $basePath = '/php-mvc-gl-ga/A3-MVC-php/actividad3/public';
29 if(strpos($uri, $basePath) === 0){
30     $uri = substr($uri, strlen($basePath));
31 }
32 if($uri == ''){
33     $uri = '/';
34 }
35
36 $router->dispatch($_SERVER['REQUEST_METHOD'], $uri);
37 >
```

4.3.7. Código del archivo inicio.php

El archivo de inicio.php muestra la bienvenida a la aplicación en conjunto de un resumen del sistema y la lista de integrantes, a su vez incluye header.php y footer.php para la estructura.

```
1 <?php include 'templates/header.php'; ?> <!--SE INCLUYE EL HEADER QUE CONTIENE LOS NAVBAR-->
2
3 <!--DISEÑOS DE LA PAGINA-->
4 <div class="position-fixed top-0 start-0 w-100 h-100"
5     style="background: url('https://d37adozyy71gtb.cloudfront.net/wp-content/uploads/2024/06/00-Portada.jpg') no-repeat center center fixed;
6     background-size: cover; opacity: 0.5; z-index: -1;"
7 </div>
8
9 <!--DIV QUE CONTIENE EL TITULO DEL PROYECTO Y UN RESUMEN DE LA APLICACION-->
10 <div class="d-flex flex-column min-vh-100">
11     <main class="container flex-grow-1 d-flex align-items-center justify-content-center text-center">
12         <div class="p-5 rounded shadow-lg bg-light text-dark">
13             <h1 class="display-4">Gestión de Libros y Autores</h1>
14             <br>
15             <h2>Bienvenido a nuestra aplicación de Gestión de Libros y Autores</h2>
16             <p>Esta es una aplicación basada en los principios de una arquitectura MVC que
17             separa responsabilidades de la aplicación, permitiéndonos tener una forma
18             sencilla y eficiente de organizar una colección de libros y la información
19             de autores.
20             Nuestra aplicación permite explorar, agregar, modificar y eliminar libros y autores
21             de manera intuitiva y sencilla. Con una interfaz moderna y funcionalidades
22             dinámicas, podemos asimilar a un gestor de una biblioteca real.
23             </p>
24             <br>
25             <!-- INTEGRANTES DEL GRUPO -->
26             <h2>Integrantes</h2>
27             <ul class="list-unstyled">
28                 <li>Alexis Damian Morales Cuasquer</li>
29                 <li>Jessica Estefania Sanchez Ugsiña</li>
30                 <li>Melanie Abigail Talavera Castillo</li>
31             </ul>
32         </div>
33     </main>
34 </div>
35
36 <?php include 'templates/footer.php'; ?> <!--SE INCLUYE EL FOOTER QUE CONTIENE EL DISEÑO Y LOS SCRIPTS NECESARIOS PARA ACCIONES DE MODALES, BOOTSTRAP, JQUERY-->
37 </div>
```



Resultados

Los resultados deben reflejarse en la facilidad y accesibilidad del ingreso de datos en nuestra página de gestión de libros y autores, además deben ser almacenados y visualizarse en una lista con la opción de Crear, editar y eliminar registros tanto para gestión de Libros y Autores.

A demás, la integración de tecnologías como php, JavaScript (con Axios), Bootstrap y MySQL es fundamental al querer mejorar la experiencia del usuario, reduciendo los tiempos de carga y facilitando una interacción más fácil con la aplicación. Al incorporar solicitudes asíncronas mediante técnicas como AJAX y Axios, se logra manipular datos sin necesidad de recargar la página, lo que a su vez mejora tanto el rendimiento como la usabilidad del sistema.

Conclusiones

- Al termino de este proyecto se concluye, que el uso de este patron MVC en php nos permite una organización adecuada de una API.
- Asimilar estas practicas nos permite familiarizarnos más con frameworks modernos.
- Aprender una organización adecuada para el desarrollo de una aplicación web.

Recomendaciones

- Se recomienda realizar un análisis detallado de los requerimientos del sistema antes de su implementación, con el fin de garantizar que todas las operaciones necesarias sean correctamente definidas y optimizadas desde la fase de planificación
- Utilizar todos los elementos sugeridos para una aplicación web dinámica y accesible, para de esta manera mejorar la experiencia del usuario, esto se puede lograr mediante el diseño de interfaces centradas en la usabilidad, asegurando que la navegación sea intuitiva y que el sistema proporcione retroalimentación visual clara en cada interacción.
- Es aconsejable llevar a cabo pruebas constantes del sistema, tanto en términos funcionales como de carga y seguridad, con el objetivo de identificar y rectificar posibles fallos antes de su implementación.